

Telecom CDR Pipeline

- 1. Kiến trúc hệ thống
- 2. Cài đặt
- 3. Pipeline & nhật ký thực thi
- 4. Kết quả
- 5. Hướng dẫn sử dụng

BÁO CÁO TRIỂN KHAI · TELECOM CDR TRÊN HADOOP ECOSYSTEM

Từ 388.800 file CDR thô đến dashboard doanh thu

Toàn bộ pipeline dữ liệu viễn thông chạy trên cluster Hadoop/Spark/Hive/MySQL tự dựng bằng Docker Compose — ghi lại đầy đủ số liệu, thời gian thực thi và các sự cố đã xử lý trong quá trình vận hành thật.

BẢN GHI CDR	DỮ LIỆU THÔ	CONTAINER	TỔNG THỜI GIAN PIPELINE
67.955.631	4,3 GB	12	~6 giờ

01 Kiến trúc hệ thống

Cluster Hadoop tối thiểu 1 master + 2 slave, dựng bằng Docker Compose trên nền image bde2020 , mở rộng thêm YARN, Hive, Spark, MySQL, Sqoop và Metabase — toàn bộ 12 container chạy trên 1 network bridge chung.

namenode	master	HDFS NameNode	heap 1.5G · mem 2.2G
datanode1 / datanode2	slave x2	HDFS DataNode	mem 1G / node
resourcemanager		YARN ResourceManager	mem 1.2G
nodemanager1		YARN NodeManager	6.1G khả dụng · mem 7G
hive-metastore-postgresql / hive-metastore / hive-server		Hive metadata + HiveServer2	heap 1.5G
spark-master / spark-worker1		Spark cluster (standalone)	worker 3.5G · 12 core
mysql		Lưu kết quả báo cáo	telecom_report
sqoop		ETL HDFS ↔ MySQL	Apache Sqoop 1.4.7 (tự build)
metabase		BI & Dashboard	5 chart

Lưu ý hạ tầng: Docker Desktop VM ban đầu chỉ cấp 7,7 GB RAM — không đủ cho khối lượng job MapReduce/Spark thực tế trên hàng trăm nghìn file nhỏ. Đã tăng lên 12.288 MiB (~11,7 GB) giữa chừng quá trình chạy để loại bỏ tình trạng NameNode/Hive/Spark liên tục bị OOM-kill.

02 Cài đặt

Khởi động toàn bộ cluster chỉ bằng một lệnh, sau đó kiểm chứng từng tầng dịch vụ trước khi nạp dữ liệu.

1 Khởi động cluster

```
docker compose up -d
```

2 Kiểm chứng HDFS thấy đủ 2 datanode

```
docker exec namenode hdfs dfs -report
```

Kết quả thực tế: **Live datanodes (2)** — namenode nhận diện đúng cả datanode1 và datanode2.

3 Kiểm chứng YARN & Hive

```
docker exec resourcemanager yarn node -list
docker exec hive-server hive -e "show databases;"
```

Các thành phần phải tự build thêm

THÀNH PHẦN	LÝ DO	CÁCH XỬ LÝ
Apache Sqoop 1.4.7	Không có image chính thức phù hợp	Dockerfile riêng trên nền <code>bde2020/hadoop-base</code> , cài tarball Sqoop + JDBC driver MySQL + <code>commons-lang-2.6.jar</code>
Python3 cho <code>simulate_ingest.py</code>	Container namenode không có Python3, không có internet egress để cài	Bind-mount <code>/home/hduser</code> từ host ra container, chạy script Python thật trên host, ETL bash đọc cùng thư mục qua mount

03 Pipeline & nhật ký thực thi

8 bước của đề bài được thực thi tuần tự trên dữ liệu thật (không phải bộ mẫu nhỏ). Log dưới đây ghi đúng số liệu và thời gian đã đo được khi chạy.

388.800

file CDR gốc sinh ra

360

file sau khi gộp theo ngày

67.955.631

dòng nạp vào bảng Hive

12

partition (year/month)

Bước 1-2 — Sinh dữ liệu & giả lập ingest

`generate_telecom_data.py` sinh **45 trạm × 360 ngày × 24 giờ = 388.800 file CSV** (~4,3 GB). `simulate_ingest.py` (Python thật) copy theo batch 45 file mỗi khoảng 1 giây từ `/home/hduser/data` sang `/home/hduser/realtime-data`; `etl_to_hdfs.sh` chạy song song mỗi 5 giây để đẩy phần mới lên HDFS.

Bước 3-4 — ETL lên HDFS & nạp bảng Hive

Đây là giai đoạn tốn nhiều công sức xử lý sự cố nhất trong toàn bộ đồ án — chi tiết đầy đủ ở nhật ký bên dưới.

~2 giờ 7 phút

Đẩy 388.800 file lên HDFS OK

`hdfs dfs -put` theo thư mục (tránh lỗi "Argument list too long" khi liệt kê từng file qua shell). Thử song song 8 luồng đã làm NameNode OOM-kill 2 lần do JVM không tôn trọng giới hạn cgroup — khắc phục bằng cách quay lại 1 luồng và set rõ `HADOOP_HEAPSIZE_MAX`.

4 lần thử

Hive INSERT OVERWRITE trên 388.800 file GC OVERHEAD LIMIT

Tăng heap Hive tuần tự **2GB → 3,5GB → 5GB → 8GB** — job vẫn fail đúng một lỗi ở đúng bước lập kế hoạch input. Kết luận: đây là giới hạn thuật toán của Hive-on-MR (Hadoop 2.7.4) khi combine quá nhiều file nhỏ, không phải thiếu RAM.

11 phút

Gộp file theo ngày: 388.800 → 360 KHẮC PHỤC SMALL-FILE PROBLEM

`scripts/merge_by_day.sh` — gộp 45 trạm × 24 giờ thành 1 file/ngày (đúng hướng xử lý đã ghi sẵn trong tài liệu dự án). Giảm số file cần Hive xử lý 1080 lần.

1-2 giờ nửa

NodeManager kẹt ở 5% suốt 24 phút THIẾU CẤU HÌNH YARN

ApplicationMaster chiếm 1.536 MB trong khi NodeManager chỉ cấu hình 2.048 MB — không còn chỗ cho mapper thực sự chạy. Tăng `yarn.nodemanager.resource.memory-mb` lên 6.144 MB.

5 phút 26 giây

Hive load thành công 67.955.631 DÒNG

16 mapper, tổng CPU 21 phút 52 giây, đọc 4,3 GB / ghi 3,6 GB Parquet. Bảng `cdx` tạo đủ 12 partition `year=2025/month=01...12`.

Bước 5 — 4 job Spark thống kê

Tất cả chạy trên `executor-memory=3000m`, worker 3,5 GB / 12 core, sau khi lần chạy đầu bị executor OOM-kill (exit 137) do bộ nhớ mặc định 1,2 GB quá nhỏ cho phép shuffle trên 68 triệu dòng.

JOB	THAM SỐ	KẾT QUẢ CHÍNH
5a — Top-K	K=10	BTS-023 dẫn đầu, 1.520.169 bản ghi
5b — Traffic theo D	D=2025-06	Call 3.120.886 bản ghi / 46,9 triệu phút
5c — Doanh thu theo dịch vụ	năm 2025	Data 613,48 tỷ đ dẫn đầu
5d — Doanh thu theo trạm	D=2025-06	45 trạm, top BTS-039 = 1,48 tỷ đ

Bước 6 — Đối chiếu HiveQL (8 phút 33 giây)

Chạy lại bằng HiveQL thuần trên bảng `cdx` đã partition — kết quả **khớp 100%** với Spark ở cả số trạm bận rộn nhất (BTS-023 = 1.520.169) lần doanh thu theo dịch vụ, xác nhận tính đúng đắn chéo giữa 2 công cụ.

Bước 7 — Sqoop export sang MySQL

Output CSV của Spark được chuẩn hoá (bỏ header lặp do nhiều part-file, bổ sung cột `report_year/report_month`) trước khi `sqoop export` — do phiên bản Sqoop thiếu sẵn `commons-lang`, phải bổ sung thủ công.

BẢNG MYSQL	SỐ DÒNG
top_callers	10
top_stations	10

BẢNG MYSQL	SỐ DÒNG
traffic_by_date	3
revenue_by_service	3
revenue_by_station	45
Tổng	71

Bước 8 – Dashboard Metabase

Dựng qua Metabase REST API (đăng nhập tài khoản admin có sẵn, đồng bộ schema, tạo 5 card, ghép vào 1 dashboard) — không cần thao tác UI thủ công.

04 Kết quả

Số liệu cuối cùng sau khi toàn bộ pipeline chạy xong trên dữ liệu thật.

Top 5 trạm bận rộn nhất	
TRẠM	BẢN GHI
BTS-023	1.520.169
BTS-010	1.519.845
BTS-005	1.518.387
BTS-042	1.517.702
BTS-017	1.517.648

Doanh thu theo dịch vụ — 2025	
DỊCH VỤ	DOANH THU (Đ)
Data	613.484.086.276
Call	154.586.600.349
SMS	4.984.386.400

Dashboard trực tiếp: <localhost:3000/dashboard/2> — 5 biểu đồ (Top trạm, Top thuê bao, doanh thu theo dịch vụ, lưu lượng theo dịch vụ, doanh thu theo trạm) dựng trên dữ liệu MySQL đã export.

05 Hướng dẫn sử dụng

Chạy lại toàn bộ hoặc từng phần của pipeline.

Chạy lại từ đầu

```
docker compose up -d
python3 scripts/generate_telecom_data.py --stations 45 --days 360 --out /home/hduser/data
bash scripts/merge_by_day.sh /home/hduser/data /home/hduser/data_merged
docker exec namenode hdfs dfs -put /home/hduser/data_merged /data/cdr
docker exec hive-server hive -f /hive/create_table.sql
```

Chạy lại 1 job Spark cụ thể

```
docker exec spark-master /spark/bin/spark-submit \
--master spark://spark-master:7077 \
--executor-memory 3000m --driver-memory 1500m \
/spark_jobs/topk_callers_stations.py --k 10
```

Export kết quả sang MySQL

```
sqoop export \  
--connect jdbc:mysql://mysql:3306/telecom_report \  
--username root --password root123 \  
--table top_stations \  
--columns station_id,total_records,total_duration_sec \  
--export-dir /sqoop_stage/top_stations.csv \  
--input-fields-terminated-by ','
```

Bài học rút ra khi vận hành ở quy mô thật

- **Small-file problem không chỉ là lý thuyết** — với 388.800 file nhỏ, Hive-on-MR đời cũ fail hoàn toàn bất kể cấp bao nhiêu RAM; gộp file theo lô là hướng xử lý bắt buộc, không phải tùy chọn.
- **JVM trong container không tự tôn trọng cgroup memory limit** (Java 8 cũ) — luôn set rõ `HADOOP_HEAPSIZE / HEAPSIZE_MAX` thay vì để mặc định.
- **YARN AM + mapper phải cùng vừa trong `yarn.nodemanager.resource.memory-mb`** — thiếu vài trăm MB là job treo vô thời hạn ở 0% chứ không báo lỗi rõ ràng.